

One important clarification: there isn't a finite list of *all* commands that works identically on every Linux distribution. Different distros and installed packages provide different utilities. Below is a practical **distro-agnostic core**, focusing on commands you can expect on most Linux systems through common POSIX/GNU tooling.

1. Getting help

man — manual pages

Bash

```
man ls
```

```
man cp
```

```
man chmod
```

Search within a manual:

Bash

```
man -k network
```

info — detailed documentation

Bash

```
info coreutils
```

--help

Most commands support:

Bash

```
ls --help
```

```
cp --help
```

```
mv --help
```

whatis — short description

Bash

```
what is ls
what is chmod
```

2. Navigation

pwd — print current directory

Bash

```
pwd
```

Example output:

```
/home/alex
```

ls — list files

Bash

```
ls
ls -l
ls -a
ls -lh
ls -lah
```

Useful options:

Command	Meaning
ls	List files
ls -l	Detailed listing
ls -a	Include hidden files
ls -h	Human-readable sizes
ls -lah	Detailed + hidden + readable sizes

cd — change directory

Bash

```
cd /etc
cd /home
cd ..
cd .
cd ~
```

Examples:

Bash

```
cd ..
```

Go to the parent directory.

Bash

```
cd ~
```

Go to your home directory.

Bash

```
cd -
```

Return to the previous directory.

3. Creating files and directories

mkdir — create directory

Bash

```
mkdir projects
mkdir documents
```

Create nested directories:

Bash

```
mkdir -p projects/linux/notes
```

touch — create an empty file/update timestamp

Bash

```
touch file.txt  
touch notes.txt
```

Multiple files:

Bash

```
touch a.txt b.txt c.txt
```

4. Copying files

cp — copy

Bash

```
cp file.txt backup.txt
```

Copy to a directory:

Bash

```
cp file.txt /tmp/
```

Copy a directory:

Bash

```
cp -r project backup
```

Preserve attributes:

Bash

```
cp -p file.txt backup.txt
```

Common combination:

Bash

```
cp -rp project backup
```

5. Moving and renaming

mv — move or rename

Rename:

Bash

```
mv old.txt new.txt
```

Move:

Bash

```
mv file.txt /tmp/
```

Move multiple files:

Bash

```
mv a.txt b.txt /tmp/
```

6. Deleting

rm — remove files

Bash

```
rm file.txt
```

Remove multiple files:

Bash

```
rm a.txt b.txt
```

Remove a directory and its contents:

Bash

```
rm -r project
```

Force removal:

Bash

```
rm -f file.txt
```

Recursive + force:

Bash

```
rm -rf project
```

Be extremely careful with `rm -rf`. There is normally no recycle bin involved.

`rmdir` — remove empty directory

Bash

```
rmdir empty_directory
```

Unlike `rm -r`, it won't remove a directory containing files.

7. Viewing files

`cat` — display file

Bash

```
cat file.txt
```

Multiple files:

Bash

```
cat file1.txt file2.txt
```

less — view large files

Bash

```
less /var/log/system.log
```

Useful keys inside less :

Space	next page
b	previous page
/word	search
n	next match
q	quit

more

Bash

```
more file.txt
```

head — beginning of file

Bash

```
head file.txt
```

First 20 lines:

Bash

```
head -n 20 file.txt
```

tail — end of file

Bash

```
tail file.txt
```

Last 20 lines:

Bash

```
tail -n 20 file.txt
```

Follow a changing file:

Bash

```
tail -f logfile.txt
```

8. Finding files

find

Find a file by name:

Bash

```
find . -name "notes.txt"
```

Find all .txt files:

Bash

```
find . -name "*.txt"
```

Find directories:

Bash

```
find . -type d
```

Find regular files:

Bash

```
find . -type f
```

Find files larger than 100 MB:

Bash

```
find . -type f -size +100M
```

Find files modified recently:

Bash

```
find . -type f -mtime -1
```

9. Searching inside files

grep

Search for text:

Bash

```
grep "error" logfile.txt
```

Case-insensitive:

Bash

```
grep -i "error" logfile.txt
```

Show line numbers:

Bash

```
grep -n "error" logfile.txt
```

Search recursively:

Bash

```
grep -r "error" .
```

Invert the match:

Bash

```
grep -v "error" logfile.txt
```

Count matches:

Bash

```
grep -c "error" logfile.txt
```

10. Text processing

These are extremely important Linux commands.

sort

Bash

```
sort names.txt
```

Reverse:

Bash

```
sort -r names.txt
```

Numeric sorting:

Bash

```
sort -n numbers.txt
```

uniq

Bash

```
uniq names.txt
```

Count consecutive duplicates:

Bash

```
uniq -c names.txt
```

Often used after sorting:

Bash

```
sort names.txt | uniq
```

wc — count

Lines:

Bash

```
wc -l file.txt
```

Words:

Bash

```
wc -w file.txt
```

Characters:

Bash

```
wc -m file.txt
```

Everything:

Bash

```
wc file.txt
```

cut — extract portions

First field separated by `:` :

Bash

```
cut -d ':' -f 1 /etc/passwd
```

First 10 characters:

Bash

```
cut -c 1-10 file.txt
```

tr — translate/delete characters

Convert lowercase to uppercase:

Bash

```
tr 'a-z' 'A-Z' < file.txt
```

Remove a character:

Bash

```
tr -d ':' < file.txt
```

diff — compare files

Bash

```
diff file1.txt file2.txt
```

cmp — compare byte-by-byte

Bash

```
cmp file1.txt file2.txt
```

11. File information

file

Determine file type:

Bash

```
file document.pdf
file image.jpg
file program
```

stat

Detailed metadata:

Bash

```
stat file.txt
```

du — disk usage

Directory size:

Bash

```
du directory
```

Human-readable:

Bash

```
du -h directory
```

Summary:

Bash

```
du -sh directory
```

df — filesystem disk space

Bash

```
df
df -h
```

12. Permissions

chmod — change permissions

Bash

```
chmod 644 file.txt
chmod 755 program
```

Symbolic form:

Bash

```
chmod u+x program
chmod u-w file.txt
chmod g+r file.txt
chmod o-r file.txt
```

chown — change owner

Bash

```
chown user file.txt
```

Owner and group:

Bash

```
chown user:group file.txt
```

chgrp — change group

Bash

```
chgrp developers file.txt
```

umask

View default permission mask:

```
Bash
```

```
umask
```

13. Users and groups

`whoami`

```
Bash
```

```
whoami
```

`id`

```
Bash
```

```
id
```

`who`

```
Bash
```

```
who
```

`w`

Show logged-in users and activity:

```
Bash
```

```
w
```

`groups`

Bash

groups

users

Bash

users

last

Show login history:

Bash

last

14. Processes

ps

Show your processes:

Bash

ps

Detailed:

Bash

ps -ef

top

Interactive process viewer:

Bash

```
top
```

kill

Terminate a process:

```
Bash
```

```
kill 1234
```

Force termination:

```
Bash
```

```
kill -9 1234
```

pgrep

Find processes by name:

```
Bash
```

```
pgrep firefox
```

pkill

Terminate processes by name:

```
Bash
```

```
pkill firefox
```

nice

Start a process with a priority:

```
Bash
```

```
nice -n 10 command
```

renice

Change priority of an existing process:

Bash

```
renice 10 -p 1234
```

15. System information

uname

Bash

```
uname  
uname -a
```

hostname

Bash

```
hostname
```

uptime

Bash

```
uptime
```

date

Bash

```
date
```

cal

Bash

```
cal
```

free

Memory information:

Bash

```
free
```

```
free -h
```

getconf

System configuration:

Bash

```
getconf PAGE_SIZE
```

16. Environment

env

Display environment:

Bash

```
env
```

printenv

Bash

```
printenv
```

Specific variable:

Bash

```
printenv HOME
```

locale

Bash

```
locale
```

17. File links

`ln` — hard link

Bash

```
ln original.txt hardlink.txt
```

Symbolic link

Bash

```
ln -s original.txt shortcut.txt
```

View links:

Bash

```
ls -l
```

18. Archives and compression

`tar`

Create archive:

Bash

```
tar -cf archive.tar files/
```

Extract:

Bash

```
tar -xf archive.tar
```

List contents:

Bash

```
tar -tf archive.tar
```

Create gzip-compressed archive:

Bash

```
tar -czf archive.tar.gz files/
```

Extract:

Bash

```
tar -xzf archive.tar.gz
```

gzip

Bash

```
gzip file.txt
```

Decompress:

Bash

```
gzip -d file.txt.gz
```

gunzip

Bash

```
gunzip file.txt.gz
```

bzip2

Bash

```
bzip2 file.txt
```

Decompress:

Bash

```
bzip2 -d file.txt.bz2
```

XZ

Bash

```
xz file.txt
```

Decompress:

Bash

```
xz -d file.txt.xz
```

19. Input/output

echo

Bash

```
echo "Hello Linux"
```

printf

Bash

```
printf "Hello Linux\n"
```

tee

Display and save output:

Bash

```
tee output.txt
```

Append:

Bash

```
tee -a output.txt
```

yes

Repeatedly output text:

Bash

```
yes
```

Usually stopped with `Ctrl+C`.

20. Redirection

These are shell operators rather than commands, but they're fundamental for using Linux commands.

Output to a file:

Bash

```
ls > files.txt
```

Append:

Bash

```
ls >> files.txt
```

Input from a file:

Bash

```
sort < names.txt
```

Standard error:

Bash

```
command 2> error.txt
```

Both output and error:

Bash

```
command > output.txt 2> error.txt
```

21. Pipes

Again, `|` is an operator, not a command.

It sends one command's output to another command.

Bash

```
ls | less
```

Bash

```
ps -ef | grep firefox
```

Bash

```
cat names.txt | sort
```

A more direct version of the last example is:

Bash

```
sort names.txt
```

22. Terminal commands

`clear`

Bash

```
clear
```

reset

Reset a corrupted terminal display:

Bash

```
reset
```

tty

Show the current terminal:

Bash

```
tty
```

stty

Display terminal settings:

Bash

```
stty
```

sleep

Pause:

Bash

```
sleep 5
```

23. Command location

which

Bash

```
which ls
which python
```

type

Bash

```
type ls
type cd
```

command

Bash

```
command -v ls
```

`command -v` is generally more useful than relying on `which` .

24. Disk and filesystem commands

mount

Show mounted filesystems:

Bash

```
mount
```

findmnt

Bash

```
findmnt
```

lsblk

Show block devices:

Bash

```
lsblk
```

blkid

Show filesystem/device identifiers:

Bash

```
blkid
```

These are common Linux utilities, although their availability can vary more than the basic POSIX commands.

25. Networking basics

ping

Bash

```
ping example.com
```

Limit packets:

Bash

```
ping -c 4 example.com
```

hostname

Bash

```
hostname
```

ip

Modern Linux networking utility:

Bash

```
ip addr
```

Show routes:

Bash

```
ip route
```

Show interfaces:

Bash

```
ip link
```

ss

Show sockets:

Bash

```
ss
```

Listening TCP/UDP sockets:

Bash

```
ss -tuln
```

curl

Transfer data from a URL:

Bash

```
curl https://example.com
```

Download:

Bash

```
curl -O https://example.com/file.txt
```

wget

Download:

Bash

```
wget https://example.com/file.txt
```

`curl` is more universally encountered on modern systems; `wget` is also very common but isn't part of the POSIX core.

26. DNS

getent

Query system databases, including hosts:

Bash

```
getent hosts example.com
```

host

Bash

```
host example.com
```

nslookup

Bash

```
nslookup example.com
```

The last two are common but may require additional packages depending on the distribution.

27. Remote access

ssh

Connect to another machine:

Bash

```
ssh user@server
```

Specify a port:

Bash

```
ssh -p 2222 user@server
```

scp

Copy files over SSH:

Bash

```
scp file.txt user@server:/tmp/
```

Copy from remote:

Bash

```
scp user@server:/tmp/file.txt .
```

sftp

Interactive file transfer:

Bash

```
sftp user@server
```

These require an SSH implementation to be installed.

28. Terminal session commands

history

Bash

```
history
```

alias

Show aliases:

Bash

```
alias
```

Create one for the current shell session:

Bash

```
alias ll='ls -l'
```

Note: `alias` itself is normally a shell builtin, so this is **not a standalone Linux executable**.

29. Time and execution

time

Measure command execution time:

Bash

```
time ls
```

timeout

Stop a command after a period:

Bash

```
timeout 10 command
```

30. Text editors

Common terminal editors include:

vi

Bash

```
vi file.txt
```

vim

Bash

```
vim file.txt
```

nano

Bash

```
nano file.txt
```

These are **not guaranteed on every Linux installation**, so don't treat them as distro-agnostic core commands.

31. POSIX-oriented core commands worth mastering

If your goal is **maximum portability across Linux distributions**, prioritize these:

cat
cd
chmod
chown
cp
cut
date
dd
df
dirname
echo
env
find
grep
head
id
kill
ln
ls
mkdir
mv
nice
printf
pwd
rm
rmdir
sed
sleep
sort
tail
tar
tee
test
touch
tr
uname
uniq
wc
xargs

A particularly important distinction:

Linux command $\neq$ shell scripting.

For example:

Bash

```
find
grep
ls
cp
mv
rm
chmod
chown
ps
kill
df
du
tar
```

are commands/utilities.

Whereas things such as:

```
if
for
while
case
function
$variable
$?
$@
```

are shell-language features, so I have intentionally **not included them**.

A good learning order

If you're learning Linux commands from scratch, I'd learn them in this order:

1. Navigation

```
pwd → ls → cd
```

2. Files

```
touch → mkdir → cp → mv → rm
```

3. Reading

cat → less → head → tail

4. Searching

find → grep

5. Text processing

sort → uniq → wc → cut → tr → diff

6. Permissions

chmod → chown → chgrp

7. Processes

ps → top → kill → pgrep

8. Storage

df → du → lsblk → mount

9. Networking

ip → ss → ping → curl

10. Archives

tar → gzip → bzip2 → xz

That gives you a strong Linux command-line foundation **without moving into shell scripting.**